Manual de Migración desde una Versión Anterior — Asistencia e Inventario

Plataforma InfoUNSA · Para el administrador de sistemas · Bases de datos MySQL

Este manual explica cómo **pasar los datos de una instalación antigua** (servidor y kioscos) a la versión actual **sin perder información**: la base de datos central de Asistencia y de Inventario, y las bases locales de cada kiosco.

> **Alcance:** bases centrales en **MySQL** (versiones del repositorio posteriores a junio de 2026, incluidas las etiquetas `v1.1.1` y `v1.2.0`). Si la instalación antigua usaba PostgreSQL, este procedimiento no aplica tal cual: consúltalo antes.

Contenido

1. Resumen y principios
2. Qué cambió entre versiones
3. Antes de empezar: diagnóstico y respaldos
4. Plan de la jornada de migración
5. Migrar Asistencia (base central)
6. Migrar Inventario (base central)
7. Actualizar los kioscos (bases locales)
8. Validación
9. Vuelta atrás (rollback)
10. Problemas frecuentes

---

1. Resumen y principios

- **Nunca se trabaja sobre la única copia de los datos.** Se hacen respaldos, y la base antigua **no se modifica**: la versión nueva usa una base nueva (Asistencia) o una copia (Inventario). Si algo falla, se vuelve a la antigua.
- **Asistencia:** la versión actual unificó las tablas. El script `scripts\migracion\migrar-asistencia.js` lee la base antigua y **copia** sus datos a una base nueva con el esquema actual. Se puede ejecutar varias veces sin duplicar datos y tiene un modo de simulación.
- **Inventario:** el esquema solo cambió de forma compatible (tablas y columnas nuevas). El script `scripts\migracion\inventario-actualizar-esquema.sql` lo actualiza **sin borrar datos y sin volver a cargar los datos de ejemplo**.

- **Kioscos:** la aplicación nueva se instala **sobre** la antigua y actualiza su base local automáticamente. Antes se debe **sincronizar todo** con el servidor antiguo.
- Las **contraseñas** de las cuentas existentes se conservan (cifradas): los usuarios entran con las mismas credenciales.

---

2. Qué cambió entre versiones

Asistencia (servidor). Las versiones anteriores guardaban los datos en **dos juegos de tablas paralelos**:

Antes	Ahora
`roles`, `sedes`, `users`, `terminals`, `attendance` (ids numéricos: cuentas y marcaciones administrativas)	`roles`, `sedes`, `users`, `terminals`, `attendance` unificadas (ids de texto)
`sync_sedes`, `sync_terminales`, `sync_usuarios` (personal con DNI), `sync_asistencias` (marcaciones de los kioscos)	Mismos datos dentro de las tablas anteriores
`sync_logs`	`sync_logs` (sin cambios)

El script de migración hace correspondencia entre ambos esquemas: une sedes y terminales con el mismo nombre/código, conserva el personal con su DNI, las cuentas con su contraseña y rol, todas las marcaciones (incluida la auditoría) y la bitácora de sincronización. Las marcaciones duplicadas (mismo `id_operacion_local`) se cargan una sola vez. Se adapta a las columnas que tenga la base antigua (`condicion`, `cargo`, `nombres`, `estado_auditoria`…).

Inventario (servidor). Se agregaron la tabla `areas`, la columna `area_id` en los movimientos, índices y restricciones de integridad (stock no negativo, cantidades válidas, operación única).

Kioscos. Las bases locales (SQLite cifrado en `%APPDATA%`) conservan el mismo nombre y mecanismo de cifrado. La aplicación nueva agrega las columnas que falten y convierte los formatos antiguos de marcaciones.

---

3. Antes de empezar: diagnóstico y respaldos

3.1. Identificar la versión instalada (opcional pero recomendado)

En el servidor del cliente, en CMD o PowerShell, desde la carpeta del repositorio nuevo:

```
mysql -u <usuario> -p <base_de_asistencia> < scripts\migracion\diagnostico-version-anterior.sql
mysql -u <usuario> -p <base_de_inventario> < scripts\migracion\diagnostico-version-anterior.sql
```

(En PowerShell, si `<` no funciona: `Get-Content scripts\migracion\diagnostico-version-anterior.sql -Raw | mysql -u <usuario> -p <base>`.)

El script **solo lee**. Muestra las tablas con su cantidad de filas y una lista de características (1 = presente):

Característica	Significado
`ASISTENCIA - tabla sync_usuarios` = 1	Es un esquema anterior: **hay que migrar** (sección 5)
`ASISTENCIA - users.dni` = 1	Ya es el esquema actual: **no migrar**, solo actualizar la aplicación
`condicion y cargo` = 2 · `estado_auditoria` = 1 · `nombres` = 1	Indican qué tan reciente es la base antigua (el migrador las maneja todas)
`INVENTARIO - areas` / `area_id` = 0	Versión previa a v1.2: el script de actualización las crea
`INVENTARIO - trigger` = 0	Falta el trigger de stock no negativo: el script lo crea

Guarda las salidas: te servirán para comparar al final.

3.2. Respaldos (obligatorios)

En el servidor **antiguo**, con las aplicaciones detenidas o sin uso:

```
mysqldump -u <usuario> -p --single-transaction --routines --triggers <base_de_asistencia> > respaldo-asistencia-ANTES-DE-MIGRAR.sql
mysqldump -u <usuario> -p --single-transaction --routines --triggers <base_de_inventario> > respaldo-inventario-ANTES-DE-MIGRAR.sql
```

Además copia los archivos `.env` / de configuración de la instalación antigua y guarda los respaldos **fuera** del servidor (otro disco o equipo). Comprueba que los archivos no estén vacíos.

---

4. Plan de la jornada de migración

Se recomienda una **ventana de mantenimiento** (por ejemplo fuera del horario de marcación) y seguir este orden:

#	Acción	Dónde	Detalle
1	**Sincronizar todos los kioscos** con el servidor antiguo hasta que la *Cola de Sync* sea 0	Cada kiosco (versión antigua)	Sección 7.1
2	Respaldar la carpeta de datos de cada kiosco	Cada kiosco	Sección 7.1
3	**Detener la API antigua** (congela los datos) y hacer los respaldos	Servidor	Sección 3.2
4	Instalar la versión nueva (clonar, dependencias, `.env`) apuntando a **bases nuevas**	Servidor	Manuales 01 y 02, pasos a–d
5	Migrar Asistencia	Servidor	Sección 5
6	Migrar Inventario	Servidor	Sección 6
7	Arrancar la API nueva y validar	Servidor	Sección 8
8	Instalar la aplicación nueva en cada kiosco	Cada kiosco	Sección 7.2
9	Validación final y conservar los respaldos y las bases antiguas varias semanas	—	Secciones 8 y 9

---

5. Migrar Asistencia (base central)

La base antigua **no se toca**. La nueva se llama, por ejemplo, `infounsa_asistencia_nueva`.

5.1. Preparar la instalación nueva siguiendo el manual técnico 01 hasta el paso d (clonar, base de datos, `.env`, dependencias). Dos diferencias:

- En el paso b crea la base **con otro nombre**, `infounsa_asistencia_nueva`, y concede permisos al usuario sobre ella.
- En el `.env` de la API, `DATABASE_URL` apunta a `infounsa_asistencia_nueva`.

5.2. Crear el esquema nuevo (vacío), en la carpeta `apps\asistencia\api`:

```
npx prisma db push
```

5.3. Simulación (no guarda nada; solo muestra qué se migraría). Desde la **raíz** del repositorio:

```
node scripts\migracion\migrar-asistencia.js --origen
"mysql://USUARIO:CLAVE@localhost:3306/infounsa_asistencia" --destino
"mysql://USUARIO:CLAVE@localhost:3306/infounsa_asistencia_nueva" --simular
```

(Sustituye `USUARIO`, `CLAVE` y los nombres de base. Si la clave tiene caracteres especiales, codifícalos en la URL: `@` → `%40`, `#` → `%23`.)

Revisa el informe: los valores `origen_*` son las filas leídas de la base antigua y `destino_*` lo que habría en la nueva. Por ejemplo, `origen_syncAtt` + `origen_attendance` menos `marcaciones_omitidas_duplicadas` debe coincidir con `destino_attendance`.

5.4. Migración real: el mismo comando **sin** `--simular`. Al terminar muestra el informe final. Puedes ejecutarlo de nuevo sin riesgo (no duplica datos); es útil si quedó algo por sincronizar.

5.5. Datos base y arranque:

```
cd apps\asistencia\api
npm run db:seed
npm run build
node .\dist\main.js
```

El seed solo **completa lo que falte** (roles, sede, terminal y las cuentas `admin` / `operador_ast`); **no modifica ni borra** datos migrados y no crea marcaciones. Luego continúa con la compilación y los servicios del manual 01 (pasos f a h), o reinicia el servicio existente apuntando al `.env` nuevo.

Qué se conserva: cuentas con su contraseña, rol y sede; personal con su DNI, nombres, condición y cargo; marcaciones con fecha y hora exactas, tipo, terminal, sede, observaciones y estado de auditoría; bitácora de sincronización.

---

6. Migrar Inventario (base central)

Para poder volver atrás, se trabaja sobre una **copia** de la base: `infounsa_inventario_nueva`.

6.1. Crear la copia (en el servidor, desde el respaldo ya hecho):

```
CREATE DATABASE infounsa_inventario_nueva CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
GRANT ALL PRIVILEGES ON infounsa_inventario_nueva.* TO 'infounsa'@'localhost';
```

```
mysql -u <usuario> -p infounsa_inventario_nueva < respaldo-inventario-ANTES-DE-
MIGRAR.sql
```

6.2. Actualizar el esquema de la copia (desde la raíz del repositorio):

```
mysql -u <usuario> -p infounsa_inventario_nueva < scripts\migracion\inventario-
actualizar-esquema.sql
```

Este script es seguro: **no borra ni sobrescribe** bienes, movimientos ni usuarios, y **no vuelve a insertar** los datos de ejemplo de `001_create_inventory_schema.sql` (por eso no se debe usar ese archivo en una base con datos reales). Se puede ejecutar más de una vez.

6.3. Configuración y arranque. Sigue el manual 02 hasta el paso d, con `DATABASE_URL` apuntando a `infounsa_inventario_nueva`, y omite el paso del esquema (ya está hecho). Ejecuta el seed solo si faltan las cuentas `admin` / `almacenero` (no cambia contraseñas existentes):

```
cd apps\inventario\api
npm run db:seed
```

6.4. Revisa las cuentas heredadas. Las instalaciones antiguas pudieron incluir **usuarios de ejemplo** con contraseñas conocidas. Revísalos y desactiva o cambia la contraseña de los que no correspondan:

```
SELECT username, role, status FROM users;
```

---

7. Actualizar los kioscos (bases locales)

7.1. Antes de actualizar cada kiosco

Con el kiosco aún en la **versión antigua** y con el servidor antiguo encendido:

1. Comprueba que el indicador diga **ONLINE** y que la **Cola de Sync** esté en **0** (espera a que se vacíe). Así todas las marcaciones del kiosco ya están en el servidor y se migrarán con él.
2. **Respalda la carpeta de datos** copiándola completa a otro lugar:
   - Asistencia: `%APPDATA%\infounsa-asistencia-desktop`
   - Inventario: `%APPDATA%\infounsa-inventario-desktop`

   (Escribe `%APPDATA%` en el Explorador de archivos o en *Ejecutar* con `Windows + R`.)
3. Si un kiosco **no pudo sincronizar** (sin red), no pasa nada: sus marcaciones pendientes se enviarán a la API nueva después de actualizarlo. El servidor ignora las que ya existan (cada marca tiene un identificador único), así que no se duplican.

7.2. Instalar la versión nueva

1. Cierra la aplicación antigua.
2. Ejecuta el instalador nuevo (`InfoUNSA-Asistencia-Setup-<versión>-x64.exe` / `InfoUNSA-Inventario-Setup-<versión>-x64.exe`; ver el manual 06) **sobre** la instalación existente. **No borres** la carpeta de datos.
3. Abre la aplicación. Al iniciar, actualiza su base local sola: agrega columnas nuevas, convierte el formato antiguo de marcaciones y **elimina las marcaciones simuladas** que las versiones de prueba generaban al primer arranque.
4. En la **configuración del servidor** (rueda dentada de la pantalla de inicio de sesión) comprueba la dirección de la API nueva: `http://<ip-del-servidor>:43000` (Asistencia) o

`:43001` (Inventario). Debe aparecer **ONLINE**.

5. Inicia sesión con las credenciales de siempre. El personal y los datos se actualizan desde el servidor al sincronizar.

> Las marcaciones antiguas que ya estaban sincronizadas viven en el **servidor** (migradas); el kiosco las consulta desde allí.

---

8. Validación

1. **Salud:** `http://localhost:43000/health` y `http://localhost:43001/health` → `status: ok`.
2. **Acceso:** inicia sesión con una cuenta antigua (administrador y operador) en cada aplicación.
3. **Cantidades:** compara con el diagnóstico y con la base antigua. En el cliente de MySQL:

   ```
   -- Asistencia (antigua):  filas de sync_asistencias + attendance
   SELECT (SELECT COUNT(*) FROM infounsa_asistencia.sync_asistencias) + (SELECT
   COUNT(*) FROM infounsa_asistencia.attendance) AS total_antiguo;
   -- Asistencia (nueva):
   SELECT COUNT(*) AS total_nuevo FROM infounsa_asistencia_nueva.attendance;
   ```

   El total nuevo puede ser menor solo por las marcaciones duplicadas que el informe indicó como omitidas.
4. **Reporte de prueba:** en Asistencia genera el reporte de un día con marcaciones antiguas y comprueba horas y nombres; en Inventario revisa el catálogo, una ficha de bien y los movimientos recientes.
5. **Kiosco:** registra una marcación de prueba o un movimiento, y comprueba que sincroniza (Cola de Sync en 0).
6. **Respaldo:** haz un respaldo de las bases nuevas y programa el respaldo diario (manual 05, sección 5).

---

9. Vuelta atrás (rollback)

Mientras conserves las bases antiguas y los respaldos, volver atrás es sencillo:

1. Detén la API nueva.
2. Vuelve a arrancar la API/aplicación antigua (apunta a las bases antiguas, que no se modificaron).

3. En los kioscos reinstala la versión anterior y, si hace falta, restaura la carpeta de datos copiada en la sección 7.1.

Conserva las bases antiguas y los respaldos **varias semanas** después de la migración antes de eliminarlos.

---

10. Problemas frecuentes

Síntoma	Causa y solución
`La base de destino no tiene el esquema actual`	Falta `npx prisma db push` sobre la base **nueva** (sección 5.2)
`La base de origen ya tiene el esquema actual`	La base antigua ya es de la versión actual: no hace falta migrar
`Access denied` / `ECONNREFUSED`	Revisa usuario, clave, puerto y que MySQL esté activo; el usuario necesita permisos sobre **ambas** bases
El informe muestra `origen_syncAtt` = 0 y `origen_attendance` > 0	La instalación antigua solo usaba las tablas administrativas; es válido, se migran igual
Aparecen menos marcaciones que antes	Revisa `marcaciones_omitidas_duplicadas`; son marcas repetidas entre los dos juegos de tablas
Un kiosco no inicia tras actualizar	Cierra la aplicación, restaura su carpeta de datos desde el respaldo y repite; si persiste, conserva la carpeta y contacta al equipo técnico
Un usuario no puede iniciar sesión	Comprueba que su cuenta existe en la base nueva (`SELECT username, status FROM users`) y que está `activo`
El script de Inventario devuelve un error de restricción	Hay datos inválidos en la base antigua (por ejemplo cantidades negativas): corrígelos en la **copia** y repite

Importante: las pruebas de este procedimiento se hicieron con bases MySQL reconstruidas a partir de las versiones del repositorio (junio a octubre de 2026). Si la instalación del cliente fue modificada a mano, ejecuta primero la **simulación** (`--simular`) y el diagnóstico, y revisa el informe antes de la migración real.